

Lab 04 – Comparing UDP and TCP Traffic with Wireshark

Transport Layer Protocol Analysis — Packet Capture, Header Inspection & Congestion Control

Name: Chase Cantrell

Course/Section: IS-3413

Date: 3/18/2025

INTRODUCTION

This lab report documents a comparative analysis of two fundamental transport layer protocols — Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) — using Wireshark packet capture analysis. The investigation examines the structural and behavioral differences between these protocols, including connection establishment procedures, header field composition, congestion control mechanisms, and their respective use cases within networked environments.

Breakpoint 1 — Loading the PCAP File in Wireshark

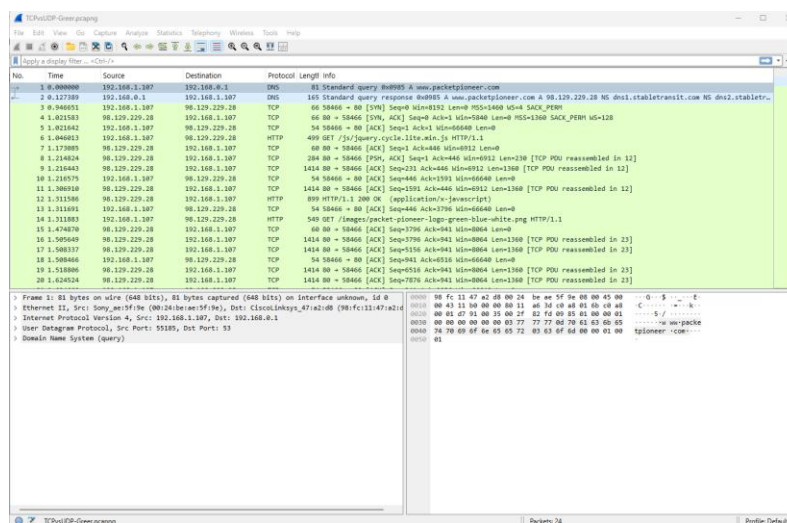


Figure 1: PCAP file loaded in Wireshark

The provided PCAP file was downloaded and imported into Wireshark for analysis. The file was opened directly through the application, presenting a capture session containing a mix of DNS (UDP) and HTTP/TCP traffic between a local host (192.168.1.107) and a remote server (98.129.229.28). The packet list pane confirmed a total of 24 captured packets, establishing the baseline dataset for subsequent protocol analysis.

Breakpoint 2 — TCP vs. UDP Overview with Microsoft Copilot

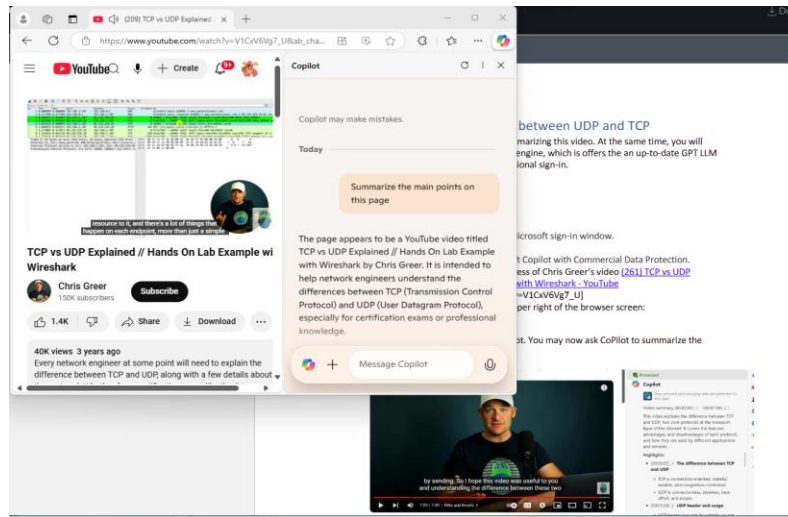


Figure 2: Microsoft Copilot summarizing the TCP vs UDP instructional video

This breakpoint involved reviewing an instructional video comparing TCP and UDP, supplemented by a Microsoft Copilot-assisted summary within Microsoft Edge. The following conceptual distinctions were derived from this activity:

TCP (Transmission Control Protocol) is a connection-oriented protocol that requires a formal connection to be established before any data transfer can occur. This process — known as the three-way handshake — relies on SYN, SYN/ACK, and ACK control messages. TCP is stateful, maintaining a continuous record of connection status, sequence numbers, and acknowledgment state throughout the session. This design prioritizes reliability and data integrity over raw throughput.

UDP (User Datagram Protocol), by contrast, is connectionless and stateless. Each datagram is transmitted independently, with no prior session establishment and no delivery confirmation. This architecture minimizes overhead, making UDP well-suited for latency-sensitive applications where occasional packet loss is preferable to transmission delay. Unlike TCP, UDP does not track delivery status or reorder out-of-sequence packets.

In summary: TCP optimizes for reliability and correctness; UDP optimizes for speed and low overhead. Each protocol serves distinct application requirements depending on whether data integrity or transmission efficiency is the higher priority.

Breakpoint 3 — Identifying UDP Traffic and Inspecting UDP Header Fields

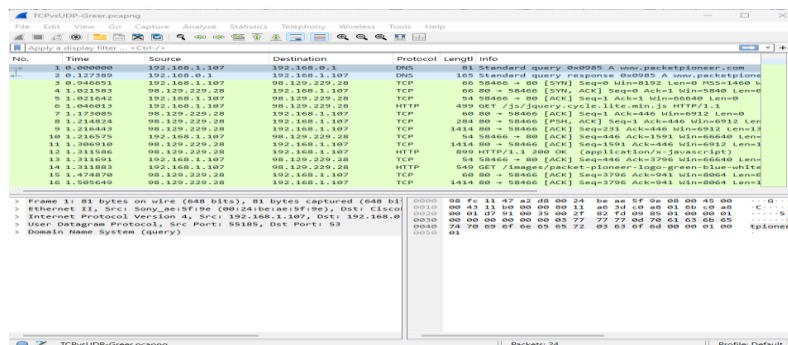


Figure 3: DNS protocol running over UDP — identified in Wireshark packet list

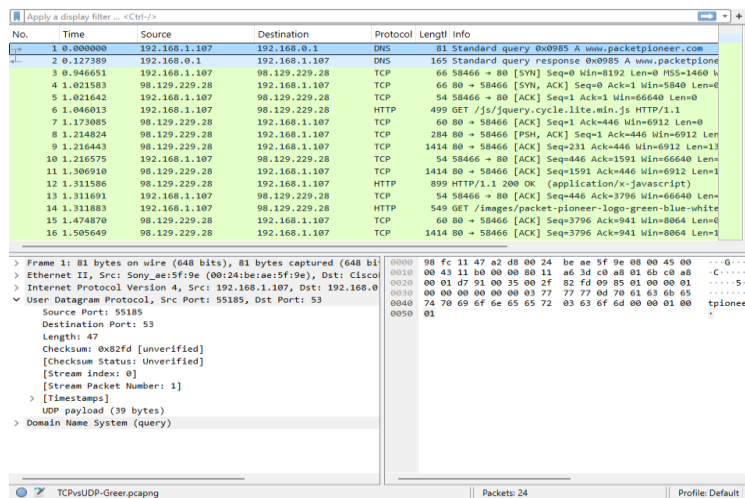


Figure 4: UDP header field breakdown for Frame 1 (DNS query)

Examination of the PCAP file revealed that DNS (Domain Name System) is the application-layer protocol operating over UDP in this capture. This was confirmed by filtering the packet list by protocol, where the first two entries — a DNS query and its corresponding response — displayed the destination domain `www.packetpioneer.com`, consistent with connectionless UDP behavior.

The UDP header, visible in the packet detail pane for Frame 1, consists of four fields. The Source Port identifies the originating port on the sending host (port 55185 in this capture). The Destination Port identifies the receiving port on the target host (port 53, the standard DNS port). The Length field specifies the total size of the UDP datagram in bytes, including both the header and payload (47 bytes here). Finally, the Checksum field provides a basic error-detection mechanism; in this capture, the checksum is listed as unverified, which is common when checksum offloading is enabled at the hardware level.

Breakpoint 4 — TCP Three-Way Handshake and TCP Header Fields

3	0.946651	192.168.1.107	98.129.229.28	TCP	66	58466 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460
4	1.021583	98.129.229.28	192.168.1.107	TCP	66	80 → 58466 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0
5	1.021642	192.168.1.107	98.129.229.28	TCP	54	58466 → 80 [ACK] Seq=1 Ack=1 Win=66640 Len=0

Figure 4: TCP three-way handshake — SYN, SYN/ACK, ACK sequence in packet list

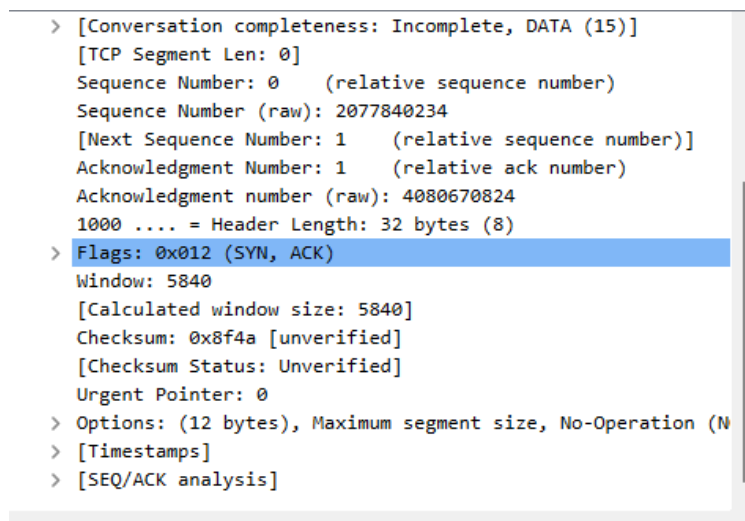


Figure 5: TCP header fields — SYN/ACK packet detail view

The TCP three-way handshake is the connection establishment mechanism that distinguishes TCP from UDP. It operates in three steps: (1) the initiating host transmits a SYN segment to signal its intent to connect; (2) the receiving host responds with a SYN/ACK, acknowledging the request and synchronizing its own sequence numbers; and (3) the initiating host completes the handshake with an ACK, confirming receipt. Only after this sequence is a TCP session considered established and data transfer may begin.

The TCP header is significantly more complex than its UDP counterpart, reflecting the protocol's additional responsibilities. Beyond the Source Port and Destination Port fields shared with UDP, the TCP header includes: the Sequence Number, which identifies the position of the first data byte in the segment within the overall byte stream; the Acknowledgment Number, which indicates the next byte the sender expects to receive, confirming all prior bytes have been received; Flags (control bits), which govern connection state transitions — including SYN, ACK, FIN, RST, and PSH; the Window Size, which advertises the amount of buffer space available, enabling flow control; the Checksum for error detection; the Urgent Pointer, which marks the end of priority data when the URG flag is active; and Options, which allow for protocol extensions such as maximum segment size (MSS) negotiation and timestamp support.

Breakpoint 5 — Congestion Control and Protocol Use Cases

TCP implements a suite of congestion control algorithms to maintain network stability under varying load conditions. These include slow start, which gradually increases the transmission rate from a conservative baseline; congestion avoidance, which moderates growth once a threshold is reached; fast retransmit, which immediately resends a segment upon detecting three duplicate acknowledgments without waiting for a timeout; and timeout-based retransmission, which handles complete packet loss scenarios. Collectively, these mechanisms make TCP well-suited for applications requiring guaranteed, ordered delivery — such as file transfers, email, and web browsing — at the cost of additional latency and overhead.

UDP, lacking these control mechanisms, operates with significantly lower overhead and latency. It is the preferred protocol for applications where speed and continuity outweigh the need for guaranteed delivery — including online gaming, live video streaming, VoIP, and DNS resolution. The reduced header size and absence of handshaking or retransmission logic make UDP particularly efficient in high-throughput, time-sensitive contexts.

Analysis of the provided PCAP file — while modest in scope at 24 packets — effectively illustrated these distinctions. The presence of both DNS (UDP) and HTTP (TCP) traffic within a single capture session provided a direct, side-by-side comparison of the two protocols in operation. Notably, the expanded TCP header relative to the UDP header reflects the additional metadata required to support reliable, stateful communication.

CONCLUSION

This lab provided a structured introduction to transport layer protocol analysis using Wireshark. Through hands-on packet capture inspection, the structural differences between TCP and UDP headers were examined in detail, and the operational implications of each protocol's design were explored in the context of real network traffic.

The primary challenge encountered was interpreting the full range of TCP header fields during initial packet analysis. The density of information presented in the Wireshark packet detail pane required methodical cross-referencing with reference materials to accurately identify the role of each field. This challenge was effectively addressed through iterative review of the capture data alongside supplementary instructional content.

The most significant technical insight from this lab was the examination of TCP's congestion control architecture. The layered approach — combining slow start, congestion avoidance, fast retransmit, and timeout-based recovery — demonstrates how TCP achieves reliable transmission across unpredictable network conditions. The competencies developed through this lab have practical applicability in network diagnostics, security analysis, and performance troubleshooting contexts, including the ability to interpret captured traffic and identify protocol-level anomalies in both home and enterprise network environments.

REFERENCES

[1] C. Greer, "TCP vs UDP Explained // Hands On Lab Example with Wireshark," YouTube, accessed Mar. 18, 2025.

Generative AI Searches:

Microsoft Copilot [GPT-4 language model], response to "What is happening in this TCP vs UDP explained video by Chris Greer," Microsoft & OpenAI. Accessed: 3/18/2025.

Collaboration:

Collaborated with Professor Maria Vicente A. Bonto-Kane during the in-class lab session on 3/18/2025.